

A Course End Project Report on

Placement Management System

Submitted by

Batch No.23

T. Akshara Reddy 25881A05CD
M. Kruthika 25881A05DC
B. Ragavulu 25881A05DM

Course Facilitator

Mr. Ganesh Desmukh

Assitant Professor

Department of Computer Science and Engineering

Vardhaman College of Engineering, Hyderabad

October2026

Contents

Abstract iii

1 Introduction 1

2 Literature Review 2

3 Objectives 3

4 Methodology 4

4.1 Block Diagram 4

4.2 System Design and Components..... 5

4.2.1 Student Management..... 5

4.2.2 Company and Job Management..... 5

4.2.3 Eligibility and Application Management..... 5

4.2.4 Record Management..... 6

4.2.5 Reporting Module 6

4.3 Working Principle..... 6

4.4 Software Implementation..... 7

5 Source Code 13

5.1 Placement Management System in Java..... 13

6 Results and Output 14

6.1 Main Menu 14

6.2 View all Students 15

6.3 Placement Application Receipt 16

6.4 Students Not yet Placed..... 17
6.5 Discussion..... 17

7 Mapping Pos and SDGs 18
7.1 Program Out comes(POs)Mapping..... 18
7.2 Sustainable Development Goals(SDGs)Mapping.....19

8 Conclusion 20

Bibliography 21

Abstract

The Placement Management System is a Java-based software application designed to automate and simplify the placement activities of a college or educational institution. The placement process involves managing student academic records, company details, job openings, eligibility requirements, interview schedules, and recruitment results. Handling these activities manually can be time-consuming and may lead to errors or difficulties in maintaining records. This system provides a centralized platform to organize placement-related information efficiently and reduce the workload of placement officers.

The proposed system provides different functionalities for students and placement officers. Placement officers can register students, maintain company information, add job opportunities, define eligibility criteria, and update placement results. Students can view available job openings, check their eligibility based on academic performance and other requirements, apply for suitable positions, and track their application status. The application can be developed using Java programming concepts such as classes, objects, inheritance, exception handling, and collections. Java Swing can be used to create a user-friendly graphical interface, while JDBC and MySQL can be used for database connectivity and permanent storage of records.

The main objective of the Placement Management System is to improve the efficiency, accuracy, and transparency of the campus recruitment process. By maintaining all placement information in an organized manner, the system reduces paperwork, minimizes data duplication, and enables quick retrieval of student and company details. It also helps placement officers monitor recruitment activities and prepare reports about student applications and selection results. The project demonstrates the practical application of Java programming and database management in solving real-world problems in educational institutions. In the future, the system can be enhanced with features such as email notifications, resume uploads, interview scheduling, and placement statistics to provide a more comprehensive recruitment management solution.

Keywords: Placement Management System, Java Programming, Campus Recruitment, Student Information Management, Company Management, Job Opportunities, Eligibility Criteria, Java Swing, JDBC, MySQL Database, Application Tracking, Placement Reports, Automation.

Chapter 1

Introduction

The Placement Management System is a Java-based software application designed to simplify and manage the placement activities of colleges and educational institutions. Campus placements help students connect with companies and explore job opportunities based on their qualifications and skills. The placement process involves maintaining student records, company details, job openings, eligibility criteria, and recruitment results. Managing these activities manually can be time-consuming, especially when many students and companies participate in the recruitment process.

The traditional placement process often relies on registers, spreadsheets, and paper-based records. This can lead to difficulties in updating information, searching for records, and maintaining accurate placement data. The Placement Management System provides a centralized platform to store and manage all placement-related information efficiently. It reduces manual work, minimizes errors, and makes information retrieval easier for students and placement officers.

The system provides different functionalities for students and placement officers. Students can view available job opportunities, check eligibility requirements, apply for suitable positions, and track their application status. Placement officers can manage student information, add company details, update job openings, verify eligibility criteria, and maintain selection results. The application can be developed using Java, with Java Swing for the graphical user interface and JDBC with MySQL for storing and managing data.

The main objective of the Placement Management System is to improve the efficiency, accuracy, and transparency of campus recruitment activities. It helps institutions organize placement information, reduce paperwork, and monitor student applications and selection results. The project demonstrates the practical use of Java programming and database management in developing a real-world application. In the future, features such as resume uploads, interview scheduling, automated notifications, and placement statistics can be added to improve the functionality of the system.

Chapter 2

Literature Review

The literature review examines existing methods and systems used to manage college placement activities. Traditionally, placement processes were handled manually using paper records, registers, and spreadsheets. These methods required considerable time and effort to maintain student information, company details, and recruitment results. They also created difficulties in updating records, searching for information, and avoiding data duplication. These limitations highlight the need for a computerized placement management system.

Existing computerized placement systems provide facilities for maintaining student profiles, storing company information, publishing job openings, and tracking recruitment activities. These systems help placement officers organize student records and manage campus recruitment more efficiently. However, some basic systems provide only limited functionalities, such as record storage and report generation, without offering convenient application tracking or automated eligibility checking.

Java-based applications provide a suitable approach for developing placement management systems because Java supports object-oriented programming, platform independence, and database connectivity. Java Swing can be used to develop interactive graphical interfaces, while JDBC allows applications to communicate with relational databases such as MySQL. These technologies support the development of applications that can perform operations such as inserting, updating, deleting, searching, and retrieving placement records.

Database management is an important part of placement systems because information about students, companies, job opportunities, and applications must be stored systematically. A relational database helps maintain relationships between different records and improves data consistency. By using appropriate database tables and keys, the system can retrieve student details, identify eligible candidates, and track application and selection statuses efficiently.

Based on the review of existing approaches, the proposed Placement Management System aims to provide a simple and organized solution for college recruitment activities. It combines student record management, company management, job eligibility checking, application tracking, and placement result management in one application. The system is intended to reduce manual effort, improve the accuracy of records, and make placement information easier to access. Additional features, such as notifications and interview scheduling, may be introduced in future versions to improve its overall usefulness.

Chapter 3

Objectives

The main objective of the Placement Management System is to simplify and automate the placement activities of colleges using Java programming. The system aims to provide an organized platform for managing student information, company details, job opportunities, and recruitment results.

- **Student Information Management:** To maintain student details, including name, roll number, branch, CGPA, contact information, and academic records.
- **Company Management:** To store and manage company profiles, job descriptions, salary packages, and recruitment requirements.
- **Eligibility Checking:** To identify eligible students based on company requirements, such as minimum CGPA, academic qualifications, and eligible branches.
- **Placement Application Management:** To enable students to view job opportunities, apply for suitable positions, and track their application status.
- **Selection Result Management:** To maintain and update the results of placement drives, including shortlisted, selected, and not-selected candidates.
- **Database Management:** To store, retrieve, update, and manage placement information efficiently using a database.
- **Reduction of Manual Work:** To minimize paperwork, reduce data-entry errors, and save time for students and placement officers.
- **Report Generation:** To generate placement reports containing student details, company information, applications, and recruitment outcomes.
- **User-Friendly Interface:** To develop a simple and easy-to-use application that allows students and placement officers to access the required features conveniently.
- **Future Enhancement:** To provide a foundation for additional features such as resume uploads, interview scheduling, automated notifications, and placement statistics.

Chapter 4

Methodology

The Placement Management System is developed using Java programming to simplify and organize college placement activities. The methodology begins with collecting and analyzing the requirements of students and placement officers. The system is designed with modules for student management, company management, job opportunities, eligibility checking, application tracking, and placement results. Java is used to implement the application logic, while Java Swing can be used to develop the graphical user interface. JDBC is used to connect the application to a MySQL database for storing and retrieving student details, company information, and placement records. The system is tested to ensure that all functions, including adding, updating, searching, and deleting records, work correctly. Finally, the application provides an organized and efficient way to manage placement information, reduce manual work, and track recruitment activities.

4.1 Block Diagram

The system consists of the following major components :

- User Module: Allows students to view job opportunities, check eligibility, apply for companies, and track application status.
- Admin Module: Enables placement officers to manage student records, company details, job openings, and placement results.
- 3Application Module: Uses Java and Java Swing to provide the user interface, process requests, validate data, and check eligibility.
- Database Module: Uses JDBC and MySQL to store, retrieve, and manage student details, company information, applications, and placement results.

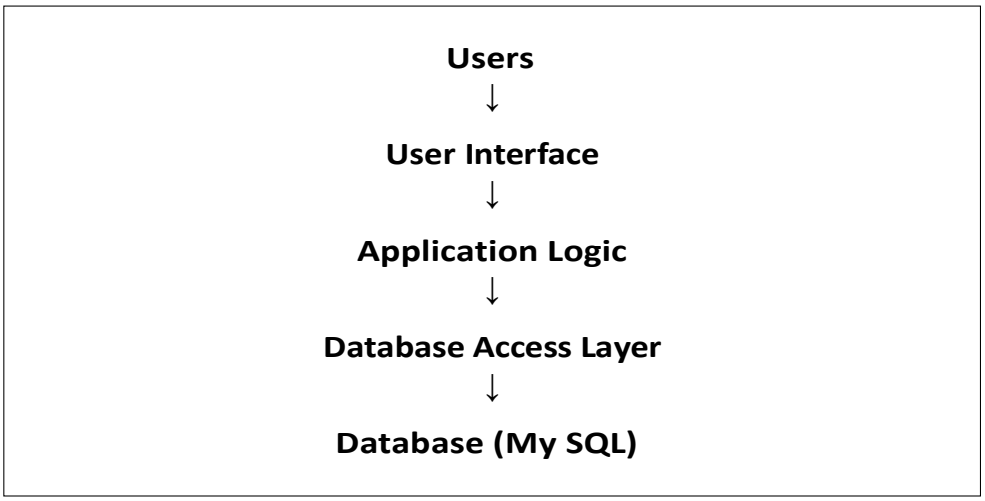


Figure4.1:BlockDiagramofPlacement Management System

4.2 System Design and Components

The Placement Management System is designed using Java, Java Swing, JDBC, and MySQL to manage student details, company information, job applications, eligibility criteria, and placement results efficiently. The main components include the User Module, User Interface, Application Logic, Database Connectivity Layer, and Database, which work together to provide an organized and user-friendly placement process.

4.2.1 Student Management

The Student Management module maintains student details such as student ID, name, branch, CGPA, email, and graduation year. It allows users to add new student records, view existing information, and search for students using their IDs. This module helps placement officers maintain organized student profiles and identify suitable candidates for different job opportunities.

4.2.2 Company and Job Management

The Company and Job Management module maintains company information and available job openings. It stores details such as company name, job role, job description, minimum CGPA requirement, and eligible branches. Placement officers can add company details and review job requirements, helping students understand available opportunities and identify jobs that match their qualifications.

4.2.3 Eligibility and Application Management

The Eligibility and Application Management module checks whether students meet the requirements specified by companies. It compares the student's branch and CGPA with the

eligibility criteria for a particular job opening. Eligible students can apply for suitable opportunities, and the system records their applications. Placement officers can update application statuses such as Applied, Shortlisted, Selected, or Not Selected to track recruitment progress.

4.2.4 Record Management

The Record Management module organizes student details, company information, job openings, and application records using Java collections. It allows users to add, search, display, and update records through the available menu options. In the current implementation, records are stored temporarily in memory while the program is running. JDBC connectivity and a MySQL database can be added in the future to maintain records permanently.

4.2.5 Reporting Module

The Reporting Module displays placement-related information maintained by the system in an organized format. It helps users view student records, available company openings, eligible candidates, submitted applications, and selection results. These details help placement officers monitor recruitment activities, review application progress, and understand placement outcomes more easily. The reports displayed depend on the features implemented and the records entered into the application.

4.3 Working Principle

The system operates through the following steps:

1. The user opens the Placement Management System and selects an operation from the main menu.

2. The placement officer adds student details such as student ID, name, branch, CGPA, and email.

3. The placement officer enters company details, job roles, minimum CGPA requirements, and eligible branches.

4. The system checks the entered information and processes the records using Java collections.

5. The system compares the student's branch and CGPA with the company's eligibility criteria to determine whether the student is eligible for a job.

6. Eligible students can apply for suitable job openings, and the placement officer updates application statuses such as Applied, Shortlisted, Selected, or Not Selected.

7. Finally, the system displays student details, company openings, applications, and placement results, helping users review recruitment progress

4.4 Software Implementation

Thesoftwaretechnologiesuseddependontheactualprojectimplementation.Therelevanttechnologiesshouldbespecifiedaccordingtotheapplicationdeveloped.

- **Programming Language:** Java
- **User Interface:** Menu-driven console interface
- **Database:** Java collections for temporary data storage; MySQL with JDBC can be added for permanent storage.
- **Development Tools:** JDK and an IDE or code editor such as VS Code, Eclipse, or IntelliJ IDEA.

The user interface allows students and placement officers to enter, view, and update placement information. In the current implementation, the Java application processes user inputs and manages student, company, and application records using Java collections. These records are stored temporarily in memory, so the current application does not directly connect to a database. In a future implementation, JDBC (Java Database Connectivity) can connect the Java application to a MySQL database. When a user enters or requests information, Java sends SQL queries through JDBC to insert, retrieve, update, or delete records in the database, and the results are displayed on the user interface.

Chapter 5

Source Code

5.1 Placement Management System in Java

The following source code implements console program that manages students, company job openings, eligibility checking, applications, and placement results.

```
import java.util.ArrayList;
import java.util.Scanner;
class Student {
    int id;
    String name, branch, email;
    double cgpa;
    Student(int id, String name, String branch,
            double cgpa, String email) {
        this.id = id;
        this.name = name;
        this.branch = branch;
        this.cgpa = cgpa;
        this.email = email;
    }
    void display() {
        System.out.println("ID: " + id
                + ", Name: " + name
                + ", Branch: " + branch
                + ", CGPA: " + cgpa
                + ", Email: " + email);
    }
}
class Company {
    String name, role, branch;
    double minCgpa;
    Company(String name, String role,
            String branch, double minCgpa) {
        this.name = name;
```

```
        this.role = role;
        this.branch = branch;
        this.minCgpa = minCgpa;
    }
    void display() {
        System.out.println("Company: " + name
                + ", Role: " + role
                + ", Eligible Branch: " + branch
                + ", Minimum CGPA: " + minCgpa);
    }
}
class Application {
    int studentId;
    String companyName;
    String status;
    Application(int studentId, String companyName) {
        this.studentId = studentId;
        this.companyName = companyName;
        this.status = "Applied";
    }
    void display() {
        System.out.println("Student ID: " + studentId
                + ", Company: " + companyName
                + ", Status: " + status);
    }
}
public class PlacementManagementSystem {
    static ArrayList<Student> students = new ArrayList<>();
    static ArrayList<Company> companies = new ArrayList<>();
    static ArrayList<Application> applications = new ArrayList<>();
    static Scanner sc = new Scanner(System.in);
    static Student findStudent(int id) {
        for (Student s : students) {
            if (s.id == id)
                return s;
        }
        return null;
    }
    static Company findCompany(String name) {
        for (Company c : companies) {
            if (c.name.equalsIgnoreCase(name))
                return c;
        }
        return null;
    }
    static void addStudent() {
        System.out.print("Enter Student ID: ");
        int id = sc.nextInt();
        sc.nextLine();
```

```
    if (findStudent(id) != null) {
        System.out.println("Student ID already exists.");
        return;
    }
    System.out.print("Enter Name: ");
    String name = sc.nextLine();
    System.out.print("Enter Branch: ");
    String branch = sc.nextLine();
    System.out.print("Enter CGPA: ");
    double cgpa = sc.nextDouble();
    sc.nextLine();
    if (cgpa < 0 || cgpa > 10) {
        System.out.println("Invalid CGPA.");
        return;
    }
    System.out.print("Enter Email: ");
    String email = sc.nextLine();
    students.add(new Student(id, name, branch, cgpa, email));
    System.out.println("Student added successfully.");
}
static void viewStudents() {
    if (students.isEmpty()) {
        System.out.println("No student records found.");
        return;
    }
    for (Student s : students)
        s.display();
}
static void addCompany() {
    sc.nextLine();
    System.out.print("Enter Company Name: ");
    String name = sc.nextLine();
    System.out.print("Enter Job Role: ");
    String role = sc.nextLine();
    System.out.print("Enter Eligible Branch: ");
    String branch = sc.nextLine();
    System.out.print("Enter Minimum CGPA: ");
    double cgpa = sc.nextDouble();
    if (cgpa < 0 || cgpa > 10) {
        System.out.println("Invalid CGPA requirement.");
        return;
    }
    companies.add(new Company(name, role, branch, cgpa));
    System.out.println("Company added successfully.");
}
static void viewCompanies() {
    if (companies.isEmpty()) {
        System.out.println("No company openings available.");
        return;
```

```
    }
    for (Company c : companies)
        c.display();
}
static void checkEligibility() {
    System.out.print("Enter Student ID: ");
    int id = sc.nextInt();
    sc.nextLine();
    Student s = findStudent(id);
    if (s == null) {
        System.out.println("Student not found.");
        return;
    }
    System.out.print("Enter Company Name: ");
    String name = sc.nextLine();
    Company c = findCompany(name);
    if (c == null) {
        System.out.println("Company not found.");
        return;
    }
    if (s.branch.equalsIgnoreCase(c.branch)
            && s.cgpa >= c.minCgpa) {
        System.out.println("Student is eligible.");
    } else {
        System.out.println("Student is not eligible.");
    }
}
static void applyForJob() {
    System.out.print("Enter Student ID: ");
    int id = sc.nextInt();
    sc.nextLine();
    Student s = findStudent(id);
    if (s == null) {
        System.out.println("Student not found.");
        return;
    }
    System.out.print("Enter Company Name: ");
    String name = sc.nextLine();
    Company c = findCompany(name);
    if (c == null) {
        System.out.println("Company not found.");
        return;
    }
    if (!s.branch.equalsIgnoreCase(c.branch)
            || s.cgpa < c.minCgpa) {
        System.out.println("Student is not eligible to apply.");
        return;
    }
    for (Application a : applications) {
```

```
            if (a.studentId == id
                    && a.companyName.equalsIgnoreCase(name)) {
                System.out.println("Application already exists.");
                return;
            }
        }
        applications.add(new Application(id, c.name));
        System.out.println("Application submitted successfully.");
    }
    static void viewApplications() {
        if (applications.isEmpty()) {
            System.out.println("No applications found.");
            return;
        }
        for (Application a : applications)
            a.display();
    }
    static void updateStatus() {
        System.out.print("Enter Student ID: ");
        int id = sc.nextInt();
        sc.nextLine();
        System.out.print("Enter Company Name: ");
        String name = sc.nextLine();
        for (Application a : applications) {
            if (a.studentId == id
                    && a.companyName.equalsIgnoreCase(name)) {
                System.out.print(
                    "Enter status (Shortlisted/Selected/Not Selected): ");
                String status = sc.nextLine();
                if (status.equalsIgnoreCase("Shortlisted")
                        || status.equalsIgnoreCase("Selected")
                        || status.equalsIgnoreCase("Not Selected")) {
                    a.status = status;
                    System.out.println("Status updated successfully.");
                } else {
                    System.out.println("Invalid status.");
                }
                return;
            }
        }
        System.out.println("Application not found.");
    }
    public static void main(String[] args) {
        int choice;
        do {
            System.out.println("\n===== PLACEMENT MANAGEMENT SYSTEM =====");
            System.out.println("1. Add Student");
            System.out.println("2. View Students");
            System.out.println("3. Add Company / Job");
```

```
            System.out.println("4. View Companies");
            System.out.println("5. Check Eligibility");
            System.out.println("6. Apply for Job");
            System.out.println("7. View Applications");
            System.out.println("8. Update Application Status");
            System.out.println("9. Exit");
            System.out.print("Enter your choice: ");
            choice = sc.nextInt();
            switch (choice) {
                case 1: addStudent(); break;
                case 2: viewStudents(); break;
                case 3: addCompany(); break;
                case 4: viewCompanies(); break;
                case 5: checkEligibility(); break;
                case 6: applyForJob(); break;
                case 7: viewApplications(); break;
                case 8: updateStatus(); break;
                case 9: System.out.println("Thank you!"); break;
                default: System.out.println("Invalid choice.");
            }
        } while (choice != 9);
        sc.close();
    }
```

Figure:5.1:Java Source code for Placement Management System

Chapter 6

Results and Output

6.1 Main Menu

When the program is executed, the following menu is displayed.

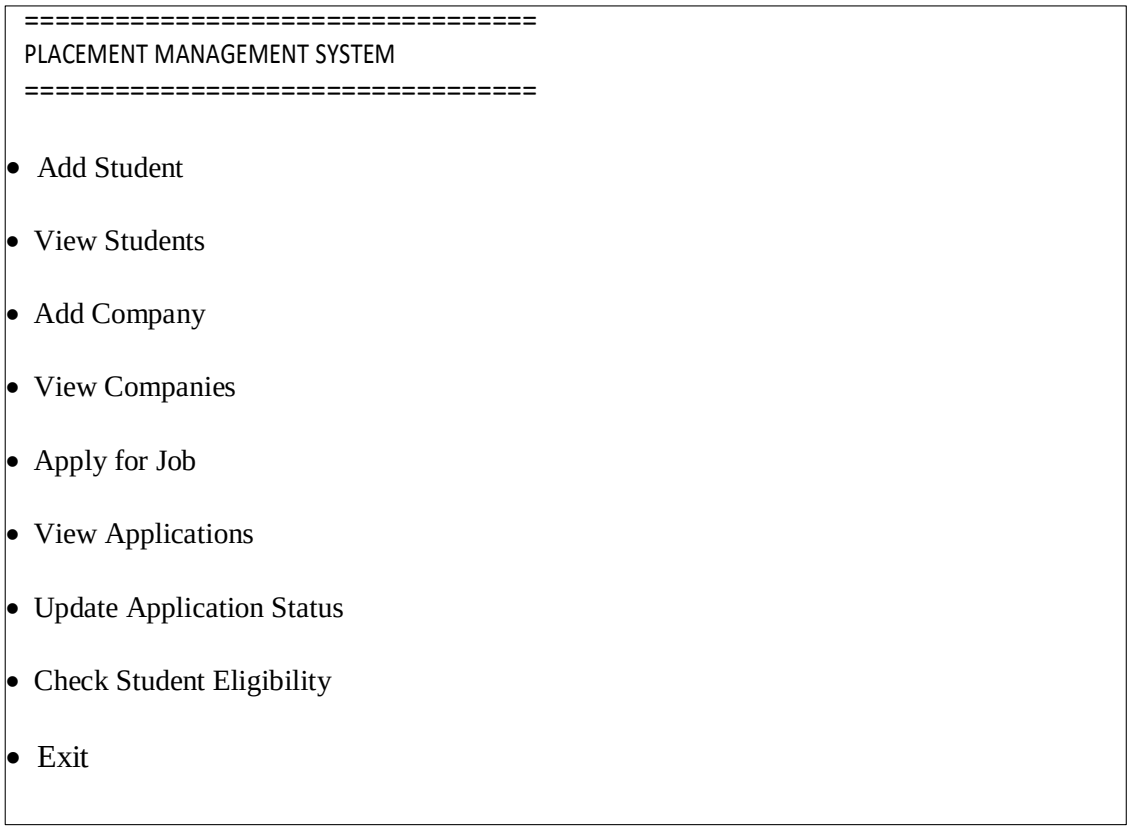


Figure6.1: Main Menu of Placement Management System

6.2 View All Students

.

```
========== ALL
STUDENTS ==========

Student ID : 101
Name       : Rahul
Branch     : CSE
CGPA       : 8.5
Email      :
rahul@gmail.com
Graduation Year : 2027

Student ID : 102
Name       : Priya
Branch     : ECE
CGPA       : 8.2
Email      :
priya@gmail.com
Graduation Year : 2027
```

Figure6.2: Placement Inventory Output

6.3 Placement Application Receipt

A Placement Application Receipt to show the details of a student's job application.

==========PLACEMENT APPLICATION RERECEIPT==========

PLACEMENT APPLICATION RECEIPT

Receipt No.	PAR-001
Date	26-10-2026

Student Details

Student ID	101
Student Name	Rahul
Branch	CSE
CGPA	8.5

Application Details

Company Name	ABC Technologies
Job Role	Software Developer
Application Status	Applied

This receipt confirms the student's job application

==========

Figure6.3: Sample receipt Output

6.4 Students Not Yet Placed

This option displays students who have not yet been selected for a placement, helping the placement officer track students who still need job opportunities.

```
====== STUDENTS NOT YET PLACED ======

Student ID : 101
Name       : Rahul
Branch     : CSE
CGPA       : 7.2
Status     : Not Placed

Student ID : 104
Name       : Priya
Branch     : ECE
CGPA       : 7.8
Status     : Not Placed

====================================
Total Unplaced Students: 2
```

Figure6.4: Sample output of students not yet placed

6.5 Discussion

The outputs of the Placement Management System show how the application manages student details, company information, and job applications. Users can add and view student records, enter company details, check student eligibility based on branch and CGPA, and apply for jobs. The system also allows users to view applications and update their status to Applied, Shortlisted, Selected, or Not Selected. These features make the placement process easier and more organized. The system uses Java ArrayLists to store data temporarily, so the records are lost when the program is close.

Chapter7

Mapping POs and SDGs

7.1 Program Outcomes (POs) Mapping

The Placement Management System applies the following outcomes to the program:

Table7.1:Mapping of Program Outcomes(POs) to Project Relevance

PO No.	Program Outcome	Relevance
PO1	Engineering Knowledge	Apply programming knowledge and Java concepts to develop the Placement Management System.
PO2	Problem Analysis	Identify placement management problems and analyze the requirements for managing students, companies, and job applications.
PO3	Design/Development of Solutions	Develop a Java-based system to add student details, manage company information, check eligibility, and track application status.
PO5	Modern Tool Usage	Use Java programming tools and an IDE to develop and execute the application.
PO6	Engineer and Society	Understand how a placement management system can help colleges organize recruitment activities and support students in finding job opportunities.

7.2 Sustainable Development Goals(SDGs)Mapping

The Placement Management System addresses the problem of managing student details, company information, eligibility criteria, and job applications manually.

Table7.2:Mapping of Sustainable Development Goals (SDGs) to Project Relevance

SDG No.	Goal	Relevance
SDG4	Quality Education	Helps students access placement opportunities and supports career development after education.
SDG8	Decent Work and Economic Growth	Helps connect eligible students with job opportunities and supports the transition from education to employment.
SDG9	Industry, Innovation and Infrastructure	Uses a software-based system to improve the organization and management of college placement activities.

Manual placement management can be time-consuming and may lead to errors in maintaining records. This project provides a simple Java-based system to organize student and company details, check student eligibility, and track application status.

Chapter 8

Conclusion

The Placement Management System is developed to simplify and organize the placement activities of a college. It provides a simple way to manage student details, company information, job opportunities, and placement applications. The system helps reduce the difficulties involved in maintaining placement records manually.

The project allows users to add and view student details such as student ID, name, branch, CGPA, email ID, and graduation year. It also allows users to add company details and view available job opportunities. These features help maintain student and company information in an organized manner.

The system also provides eligibility checking based on the student's branch and CGPA requirements specified by the company. Eligible students can apply for jobs, and the placement officer can view applications and update their status as Applied, Shortlisted, Selected, or Not Selected. This makes it easier to track the progress of placement applications.

This project is implemented using Java programming concepts, object-oriented programming, and `ArrayList` collections for temporary data storage. The menu-driven interface makes the system simple to understand and operate. However, the current version does not store data permanently, so records are lost when the program is closed.

In conclusion, the Placement Management System provides a useful foundation for managing college placement activities efficiently. In the future, the system can be improved by integrating a MySQL database using JDBC, developing a graphical user interface, adding secure login facilities, and generating placement reports. These improvements can make the system more reliable, user-friendly, and suitable for real-world college placement management.

Bibliography

1. Herbert Schildt, Java: The Complete Reference, McGraw Hill Education.
2. Oracle, Java Documentation. Available at: https://docs.oracle.com/en/java/.
3. Oracle, Java Collections Framework. Available at: https://docs.oracle.com/javase/8/docs/technotes/guides/collections/.
4. W3Schools, Java Tutorial. Available at: https://www.w3schools.com/java/.
5. Geeks for Geeks, Java Programming Language. Available at: https://www.geeksforgeeks.org/java/.